



CENTRO EDUCATIVO DEPARTAMENTAL MUNICIPAL

MÍGUELA RODRÍGUEZ ARAUJO — Luque, Paraguay

Java desde cero

Programación Orientada a Objetos, clases, objetos y primeros programas

Laboratorio · Gabinete de Informática — 3º BATIN — Etapa 2

Prof. Lic. Daniela Ferreira

"Saber, hacer y transformar"

¿Qué vamos a lograr hoy?

- 1 Entender qué es la Programación Orientada a Objetos (POO) y por qué la usamos
- 2 Distinguir con claridad entre clase y objeto, atributo y método
- 3 Escribir y ejecutar el primer programa en Java dentro del entorno BATIN_PORTABLE
- 4 Declarar variables, usar tipos de datos primitivos y leer datos por teclado con Scanner
- 5 Aplicar operadores aritméticos, de asignación y de incremento/decremento en código real

01

Fundamentos de P00

Antes de escribir una sola línea de Java, necesitamos el vocabulario y la forma de pensar de la Programación Orientada a Objetos.

¿Qué es un lenguaje de programación?

- **Imperativo:** una secuencia de instrucciones paso a paso (ej.: pseudocódigo clásico)
- **Funcional:** todo se expresa como funciones que reciben una entrada y devuelven una salida
- **Lógico:** se describen reglas y hechos, y el sistema deduce las respuestas
- **Orientado a objetos:** el mundo se modela como objetos que tienen datos y comportamiento propio — este es el que vamos a usar

¿Por qué pensar en objetos?

- Bit y Techí están armando el sistema de la feria estudiantil. Tienen que manejar estudiantes, puestos, productos y ventas.
- Si programan todo como una lista larga de instrucciones sueltas, cada vez que algo cambia (por ejemplo, agregar un dato al estudiante) hay que tocar el código en muchos lugares distintos.
- La POO propone agrupar los datos y las acciones que le corresponden a una misma "cosa" del mundo real dentro de una unidad: el objeto.
- Así, todo lo que tiene que ver con "Estudiante" vive junto, y el resto del programa no necesita saber cómo está armado por dentro.

Objeto, clase, atributo, método

- **Objeto:** una entidad concreta del programa que representa algo del mundo real (un estudiante, un producto)
- **Clase:** el molde o plano que describe cómo va a ser cada objeto de ese tipo
- **Atributo:** un dato que guarda el objeto (nombre, precio, cédula)
- **Método:** una acción que el objeto puede realizar (saludar(), calcularTotal())

Clase vs. Objeto: el plano y la casa

CLASE

El plano, el molde. Define QUÉ atributos y QUÉ métodos va a tener cada objeto — pero todavía no es ningún objeto en concreto.

```
class Estudiante {  
  
    String nombre;  
    String curso;  
  
    void saludar() {  
        ...  
    }  
}
```

OBJETO

La casa construida, concreta. Es una instancia real de la clase, con VALORES propios en sus atributos.

```
Estudiante e1 = new Estudiante();  
e1.nombre = "Gabriela";  
e1.curso = "3° BATIN";  
  
Estudiante e2 = new Estudiante();  
e2.nombre = "Marcos";
```

Encapsulación, abstracción, herencia y polimorfismo

- **Encapsulación:** el objeto protege sus datos; solo se accede a ellos a través de sus propios métodos
- **Abstracción:** mostramos solo lo esencial del objeto y ocultamos los detalles internos de implementación
- **Herencia:** una clase puede heredar atributos y métodos de otra más general (ej.: Producto → ProductoAlimenticio)
- **Polimorfismo:** objetos de distintas clases pueden responder de forma distinta al mismo mensaje/método

De la teoría al código

- Ya tenemos el vocabulario: clase, objeto, atributo, método, y los cuatro pilares.
- En la práctica, cuando escribimos Java, cada 'class' que definimos es literalmente el plano del que hablamos.
- A partir de ahora vamos a ver cómo se ve todo esto escrito en código Java real, empezando por la estructura mínima de un programa.

02

Introducción a Java

Java es el lenguaje orientado a objetos que vamos a usar durante todo el año para construir el sistema propio de cada equipo.

¿Por qué Java para nuestro proyecto?

- **Orientado a objetos:** aplica directamente todo lo que acabamos de ver: clases, objetos, herencia
- **Multiplataforma:** el mismo programa compilado corre en Windows, Linux o Mac sin cambios
- **Tipado fuerte:** cada variable tiene un tipo definido, lo que ayuda a detectar errores antes de ejecutar
- **Uso real en el proyecto:** vamos a conectarlo a SQLite vía JDBC y construir interfaces con Swing

BATIN_PORTABLE: todo lo que ya tenemos listo

- JDK (Temurin 17): el compilador y el entorno de ejecución de Java
- Visual Studio Code portable: el editor de código que vamos a usar en todas las clases
- SQLite + DB Browser: para cuando conectemos nuestros programas a una base de datos
- Todo funciona sin instalación y sin conexión a internet, desde la carpeta BATIN_PORTABLE

Estructura mínima de un programa Java

- Todo programa Java vive dentro de una clase — el nombre de la clase debe coincidir con el nombre del archivo (.java).
- El método main es el punto de entrada: es lo primero que se ejecuta cuando corremos el programa.

```
public class MiPrimerPrograma {  
  
    public static void main(String[] args) {  
        // acá va el código que se ejecuta  
    }  
  
}
```

"Hola, feria!" — nuestro primer programa completo

- `System.out.println()` imprime un texto en la consola y salta de línea al final.

```
public class HolaFeria {  
  
    public static void main(String[] args) {  
        System.out.println("¡Hola, Feria Estudiantil!");  
    }  
  
}
```

¿Qué es una variable?

- Una variable es un espacio con nombre en la memoria donde guardamos un dato que puede cambiar durante la ejecución del programa.
- En Java, toda variable tiene un tipo de dato que se define al declararla — y ese tipo no cambia después.
- Declarar una variable es decirle a Java: "reservá espacio para este tipo de dato, con este nombre".

Tipos de datos primitivos en Java

Tipo	Guarda	Ejemplo
int	números enteros	int edad = 16;
double	números decimales	double precio = 15000.50;
boolean	verdadero o falso	boolean activo = true;
char	un solo carácter	char inicial = 'G';
String	cadena de texto (no es primitivo, pero se usa igual)	String nombre = "Gabriela";

Declarar y usar variables en código

```
public class DatosEstudiante {  
  
    public static void main(String[] args) {  
        String nombre = "Gabriela";  
        int edad = 16;  
        double promedio = 8.75;  
        boolean esRepresentante = true;  
  
        System.out.println(nombre + " tiene " + edad + " años");  
    }  
}
```

Scanner: leer lo que escribe el usuario

- **`import java.util.Scanner;;`** se escribe al principio del archivo, para poder usar la clase Scanner
- **`Scanner entrada = new Scanner(System.in);`** crea un objeto Scanner conectado al teclado
- **`entrada.nextLine();`** espera a que el usuario escriba y presione Enter, y devuelve lo que escribió

Métodos de Scanner según el tipo de dato

Método	Lee
nextInt()	un número entero (int)
nextDouble()	un número decimal (double)
nextBoolean()	un valor verdadero/falso
nextLine()	una línea completa de texto
next()	una sola palabra (sin espacios)

Ejemplo completo con Scanner

```
import java.util.Scanner;

public class SaludoPersonalizado {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.println("¿Cómo te llamas?");
        String nombre = entrada.nextLine();

        System.out.println("¡Hola, " + nombre + "!");
    }
}
```

Operadores aritméticos en Java

Operador	Significado	Ejemplo
+	adición	$6 + 3 \rightarrow 9$
-	sustracción	$6 - 3 \rightarrow 3$
*	multiplicación	$6 * 3 \rightarrow 18$
/	división	$6 / 3 \rightarrow 2$
%	módulo (resto)	$7 \% 3 \rightarrow 1$

Asignación e incremento/decremento

- **Asignación compuesta:** `+=`, `-=`, `*=`, `/=`, `%=` — combinan una operación con la asignación (`x += 1`; equivale a `x = x + 1`;)
- **Incremento y decremento:** `++` y `--` suman o restan 1 a una variable
- **Prefijo (`++x`):** incrementa primero, y usa el nuevo valor en la expresión
- **Sufijo (`x++`):** usa el valor actual en la expresión, y recién después incrementa

Prefijo vs. sufijo en código

- Con test = 5, ++test da 6. Con test = 5, test-- usado en una expresión usa 5 y luego queda en 4.

```
public class Operadores {  
    public static void main(String[] args) {  
        int test = 5;  
        ++test;  
        System.out.println(test);    // 6  
  
        int otro = 5;  
        otro--;  
        System.out.println(otro);    // 4  
    }  
}
```

String: unir texto con el operador +

- Un String es un objeto que representa una secuencia de caracteres.
- El operador + entre cadenas de texto las une — a esto se lo llama concatenación.

```
public class DatosCompletos {  
    public static void main(String[] args) {  
        String nombre = "David";  
        String apellido = "Williams";  
        System.out.println("Mi nombre es " + nombre + " " + apellido);  
    }  
}
```


Tarea

Escribí un programa Java que:

- 1 Capture el nombre y apellido del usuario, y su edad actual (usando Scanner)
- 2 Imprima su nombre completo
- 3 Calcule y muestre cuál era su edad hace 3 años
- 4 Calcule y muestre cuál será su edad dentro de 144 meses (usá operadores de asignación)

Lo que viene

- Hoy sumamos: fundamentos de POO (clase, objeto, atributo, método) + sintaxis Java (variables, tipos, Scanner, operadores).
- La próxima clase de Laboratorio seguimos con estructuras de control: condicionales y ciclos (for, while).
- Después: arreglos en Java, y la conexión de todo esto con las clases del sistema propio de cada equipo.



CENTRO EDUCATIVO DEPARTAMENTAL MUNICIPAL

MÍGUELA RODRÍGUEZ ARAUJO — Luque, Paraguay

Java desde cero — Más ejercicios

Segunda tanda: operadores, Scanner y concatenación aplicados

Laboratorio · Gabinete de Informática — 3º BATIN — Etapa 2

Complemento de la práctica interactiva "Java desde cero"

"Saber, hacer y transformar"

Total de la compra

Completá el programa para que lea el precio de un producto (double) y la cantidad de unidades (int), y calcule el total a pagar.

```
import java.util.Scanner;

public class TotalCompra {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.println("¿Cuál es el precio del producto?");
        double precio = /* completá acá */;

        System.out.println("¿Cuántas unidades vas a llevar?");
        int cantidad = /* completá acá */;

        double total = /* completá acá: precio por cantidad */;
        System.out.println("Total a pagar: Gs. " + total);
    }
}
```

Total de la compra — Solución

```
import java.util.Scanner;

public class TotalCompra {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.println("¿Cuál es el precio del producto?");
        double precio = entrada.nextDouble();

        System.out.println("¿Cuántas unidades vas a llevar?");
        int cantidad = entrada.nextInt();

        double total = precio * cantidad;
        System.out.println("Total a pagar: Gs. " + total);
    }
}
```

💡 Al multiplicar un double por un int, Java convierte el int a double: el resultado siempre es double.

Vuelto

Completá el programa para que lea el total a pagar y el monto con el que paga el cliente (ambos double), y calcule el vuelto.

```
import java.util.Scanner;

public class Vuelto {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.println("¿Cuánto es el total a pagar?");
        double total = /* completá acá */;

        System.out.println("¿Con cuánto paga el cliente?");
        double pago = /* completá acá */;

        double vuelto = /* completá acá */;
        System.out.println("Vuelto: Gs. " + vuelto);
    }
}
```

Vuelto — Solución

```
import java.util.Scanner;

public class Vuelto {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.println("¿Cuánto es el total a pagar?");
        double total = entrada.nextDouble();

        System.out.println("¿Con cuánto paga el cliente?");
        double pago = entrada.nextDouble();

        double vuelto = pago - total;
        System.out.println("Vuelto: Gs. " + vuelto);
    }
}
```

💡 El vuelto siempre se calcula como lo que pagó el cliente menos el total — el orden de la resta importa.

Promedio de notas

Completá el programa para que calcule el promedio de las tres notas declaradas e imprima el resultado.

```
public class PromedioNotas {  
    public static void main(String[] args) {  
        double n1 = 8.0;  
        double n2 = 7.0;  
        double n3 = 9.0;  
  
        double promedio = /* completá acá */;  
        System.out.println("Promedio: " + promedio);  
    }  
}
```


Promedio de notas — Solución

```
public class PromedioNotas {  
    public static void main(String[] args) {  
        double n1 = 8.0;  
        double n2 = 7.0;  
        double n3 = 9.0;  
  
        double promedio = (n1 + n2 + n3) / 3;  
        System.out.println("Promedio: " + promedio);  
    }  
}
```

💡 Los paréntesis son necesarios: sin ellos, Java resolvería primero $n3 / 3$ por precedencia de operadores, y el resultado sería otro.

Ticket del puesto

Completá la línea que arma la variable ticket, combinando producto, cantidad y precio, para que se imprima: "Empanada x2 - Gs. 3000.0"

```
public class TicketPuesto {  
    public static void main(String[] args) {  
        String producto = "Empanada";  
        double precio = 3000.0;  
        int cantidad = 2;  
  
        String ticket = /* completá acá */;  
        System.out.println(ticket);  
    }  
}
```

Ticket del puesto — Solución

```
public class TicketPuesto {  
    public static void main(String[] args) {  
        String producto = "Empanada";  
        double precio = 3000.0;  
        int cantidad = 2;  
  
        String ticket = producto + " x" + cantidad + " - Gs. " + precio;  
        System.out.println(ticket);  
    }  
}
```

💡 Se puede armar un String completo en una variable antes de imprimirlo — no hace falta que la concatenación esté directamente dentro del `println`.

Reparto en filas

Hay que ubicar a los estudiantes en filas de un tamaño fijo. Completá el programa para calcular cuántas filas completas se forman y cuántos estudiantes quedan sueltos.

```
public class RepartoFilas {  
    public static void main(String[] args) {  
        int totalEstudiantes = 23;  
        int porFila = 5;  
  
        int filas = /* completá acá: división entera */;  
        int sobran = /* completá acá: resto */;  
  
        System.out.println("Filas completas: " + filas);  
        System.out.println("Estudiantes que sobran: " + sobran);  
    }  
}
```

Reparto en filas — Solución

```
public class RepartoFilas {  
    public static void main(String[] args) {  
        int totalEstudiantes = 23;  
        int porFila = 5;  
  
        int filas = totalEstudiantes / porFila;  
        int sobran = totalEstudiantes % porFila;  
  
        System.out.println("Filas completas: " + filas);  
        System.out.println("Estudiantes que sobran: " + sobran);  
    }  
}
```

💡 La división entera (/) da el cociente sin decimales, y el módulo (%) da el resto de esa misma división. Juntos resuelven cualquier problema de "repartir en grupos".

Contador de ventas

Un puesto registra tres ventas y después le anulan una. Completá el programa usando ++ y -- para llegar al total neto correcto.

```
public class ContadorVentas {
    public static void main(String[] args) {
        int ventas = 0;

        // Se registran tres ventas
        /* completá acá: incrementá ventas tres veces */

        // Se anula una venta
        /* completá acá: decrementá ventas una vez */

        System.out.println("Ventas netas del día: " + ventas);
    }
}
```

Contador de ventas — Solución

```
public class ContadorVentas {  
    public static void main(String[] args) {  
        int ventas = 0;  
  
        // Se registran tres ventas  
        ventas++;  
        ventas++;  
        ventas++;  
  
        // Se anula una venta  
        ventas--;  
  
        System.out.println("Ventas netas del día: " + ventas);  
    }  
}
```

💡 Cada ++ o -- modifica la variable directamente, sin necesidad de reasignarla con =. Se pueden encadenar tantos como se necesiten.

Recordatorio

- Estos 6 ejercicios (7 a 12) se suman a los 6 originales en la práctica interactiva "Java desde cero".
- Podés escribir tu propio código, ejecutarlo y ver la salida real directamente en esa página — no hace falta VS Code para esto.
- Cuando tengas dudas, tocá "Ver solución" en cada ejercicio antes de mirar esta presentación.